

SQL Interview Q&A — Worked Examples

40 questions, answered example-by-example with queries and outputs · Basic → Advanced → Scenario · Capital-markets flavored

Reference Tables

employees

emp_id	name	dept	mgr	salary
1	Asha	Operations	NULL	95000
2	Ben	Operations	1	62000
3	Carol	Settlements	1	68000
4	Dev	Settlements	3	54000
5	Ella	Reconciliation	1	71000
6	Faisal	Reconciliation	5	48000

clients

client_id	client_name	country
1	Meridian Capital	UK
2	Nova Funds	US
3	Orion AM	India
4	Pallas LLP	US

trades

trade_id	client	symbol	qty	price	date
101	1	AAPL	100	180	01-05
102	1	MSFT	50	400	01-06
103	2	AAPL	200	182	01-06
104	3	TSLA	75	210	01-07
105	2	MSFT	30	405	01-08

Part A — Basic (1–12)

Q1. Select all employees earning more than 60,000.

```
SELECT name, salary FROM employees WHERE salary > 60000;
```

Output: Asha 95000, Ben 62000, Carol 68000, Ella 71000

Q2. Find the 3 highest-paid employees.

```
SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 3;
```

Output: Asha 95000, Ella 71000, Carol 68000

(SQL Server: `SELECT TOP 3 ...`; Oracle: `FETCH FIRST 3 ROWS ONLY`.)

Q3. List the distinct departments.

```
SELECT DISTINCT dept FROM employees;
```

Output: Operations, Settlements, Reconciliation

Q4. Find employees whose name starts with "C".

```
SELECT name FROM employees WHERE name LIKE 'C%';
```

Output: Carol — `%` = any characters, `_` = exactly one character.

Q5. Which employees have no manager?

```
SELECT name FROM employees WHERE manager_id IS NULL; -- Asha
```

Why not `= NULL`? Because NULL means "unknown" — no value equals it. Comparisons to NULL yield *unknown*, so `WHERE manager_id = NULL` returns zero rows. Always use `IS NULL` / `IS NOT NULL`.

Q6. Count employees per department.

```
SELECT dept, COUNT(*) AS headcount FROM employees GROUP BY dept;
```

dept	headcount
Operations	2
Settlements	2
Reconciliation	2

Q7. What's the difference between COUNT(*), COUNT(col), and COUNT(DISTINCT col) ?

- COUNT(*) — every row, including NULLs. On employees = 6.
- COUNT(manager_id) — non-NULL values only. = 5 (Asha's is NULL).
- COUNT(DISTINCT dept) — unique non-NULL values. = 3.

Q8. Show departments whose average salary exceeds 60,000.

```
SELECT dept, AVG(salary) AS avg_sal
FROM employees GROUP BY dept HAVING AVG(salary) > 60000;
```

Output: Operations 78500, Settlements 61000

WHERE vs HAVING: WHERE can't be used here because the filter is on an *aggregate* — WHERE runs before grouping, HAVING runs after.

Q9. Total value (notional) traded per symbol.

```
SELECT symbol, SUM(qty*price) AS notional
FROM trades GROUP BY symbol ORDER BY notional DESC;
```

symbol	notional
AAPL	54400
MSFT	32150
TSLA	15750

AAPL = 100×180 + 200×182 = 54,400.

Q10. Employees hired... sorry, order employees by salary, lowest first, but show NULL-manager rows last.

```
SELECT name, salary FROM employees
ORDER BY salary ASC; -- add NULLS LAST when sorting a nullable column
```

Postgres/Oracle support ORDER BY manager_id NULLS LAST. In MySQL, NULLs sort first on ASC by default.

Q11. Bucket employees into salary bands.

```
SELECT name, salary,
       CASE WHEN salary >= 80000 THEN 'Band A'
            WHEN salary >= 60000 THEN 'Band B'
            ELSE 'Band C' END AS band
FROM employees;
```

name	salary	band
Asha	95000	Band A
Carol	68000	Band B
Faisal	48000	Band C

Q12. Replace NULL manager with 0 in the output.

```
SELECT name, COALESCE(manager_id, 0) AS mgr FROM employees;
```

COALESCE returns the first non-NULL argument. Asha's mgr shows as 0.

Part B — Joins & Subqueries (13–24)

Q13. Show each trade with the client's name.

```
SELECT t.trade_id, c.client_name, t.symbol, t.qty
FROM trades t INNER JOIN clients c ON t.client_id = c.client_id;
```

Returns 5 rows. Pallas LLP (client 4) has no trades, so INNER JOIN excludes it.

Q14. Which clients have never traded?

```
SELECT c.client_name
FROM clients c LEFT JOIN trades t ON c.client_id = t.client_id
WHERE t.trade_id IS NULL;
```

Output: Pallas LLP

This is the **anti-join** pattern — LEFT JOIN keeps all clients, and the unmatched ones have NULL trade columns. In corporate actions this is how you find *unsettled* or *unmatched* instructions.

Q15. Count trades per client, including clients with zero.

```
SELECT c.client_name, COUNT(t.trade_id) AS num_trades
FROM clients c LEFT JOIN trades t ON c.client_id = t.client_id
GROUP BY c.client_name;
```

client_name	num_trades
Meridian Capital	2
Nova Funds	2
Orion AM	1
Pallas LLP	0

Note `COUNT(t.trade_id)` not `COUNT(*)` — the latter would count the NULL row as 1 for Pallas.

Q16. Show each employee next to their manager's name.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

A **self join** — the table joined to itself with two aliases. Ben→Asha, Dev→Carol, Faisal→Ella, Asha→NULL.

Q17. Explain INNER vs LEFT vs RIGHT vs FULL with row counts.

- **INNER** — only matching rows in both. (`trades` × `clients` = 5)
- **LEFT** — all clients + matches; Pallas gets NULLs. (`rows` ≥ left table)
- **RIGHT** — all trades + matches (mirror of LEFT).
- **FULL OUTER** — everything from both, NULLs where no match.
- **CROSS** — every combination: 4 clients × 5 trades = 20 rows.

Q18. Find employees earning above the company average.

```
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Avg ≈ 66,333 → Asha, Carol, Ella

The inner query is a **scalar subquery** — returns a single value used by the outer WHERE.

Q19. Find the top earner in each department.

```
SELECT name, dept, salary FROM employees e
WHERE salary = (SELECT MAX(salary) FROM employees e2 WHERE e2.dept = e.dept);
```

Asha (Ops 95000), Carol (Settlements 68000), Ella (Recon 71000)

This is a **correlated subquery** — the inner query references the outer row (`e.dept`) and re-runs per row.

Q20. List clients who have at least one trade — using EXISTS.

```
SELECT client_name FROM clients c
WHERE EXISTS (SELECT 1 FROM trades t WHERE t.client_id = c.client_id);
```

Returns Meridian, Nova, Orion. `EXISTS` stops at the first match — often faster than `IN` for large subqueries.

Q21. IN vs EXISTS vs JOIN — which to use?

- **IN** — clean for a small, static list of values.
- **EXISTS** — best for correlated existence checks; short-circuits; NULL-safe.
- **JOIN** — when you also need columns from the other table.

Trap: `NOT IN` with a subquery that can return NULL gives wrong/empty results — prefer `NOT EXISTS` .

Q22. Clients who traded both AAPL and MSFT.

```
SELECT client_id FROM trades WHERE symbol='AAPL'
INTERSECT
SELECT client_id FROM trades WHERE symbol='MSFT';
```

Output: 1 (Meridian), 2 (Nova)

Or with GROUP BY: `HAVING COUNT(DISTINCT symbol) = 2` over those two symbols.

Q23. Clients who traded AAPL but never TSLA.

```
SELECT client_id FROM trades WHERE symbol='AAPL'
EXCEPT
SELECT client_id FROM trades WHERE symbol='TSLA';  -- 1, 2
```

`EXCEPT` (Oracle: `MINUS`) = rows in the first set not in the second.

Q24. UNION vs UNION ALL — show the difference.

```
SELECT country FROM clients      -- UK, US, India, US
UNION      SELECT country FROM clients;  -- UK, US, India  (dedup)
-- UNION ALL keeps duplicates: UK, US, India, US, UK, US, India, US
```

Use `UNION ALL` when you know there are no dupes (or want them) — it skips the expensive de-duplication sort.

Part C — Window Functions & CTEs (25–33)

Q25. Rank employees by salary. Show ROW_NUMBER, RANK, DENSE_RANK together.

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,
       RANK() OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM employees;
```

If two salaries tied at rank 2: ROW_NUMBER → 2,3; RANK → 2,2,4 (skips 3); DENSE_RANK → 2,2,3 (no gap). ROW_NUMBER is always unique.

Q26. Find the 2nd highest salary. (asked in ~every SQL interview)

```
-- Method 1: DENSE_RANK
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rk FROM employees
) x WHERE rk = 2;           -- 71000

-- Method 2: subquery
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Method 3: OFFSET
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

For the **Nth** highest, change to `rk = N` (DENSE_RANK handles ties correctly).

Q27. Highest-paid employee *within each department*.

```
WITH ranked AS (
  SELECT name, dept, salary,
         ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn
  FROM employees
)
SELECT name, dept, salary FROM ranked WHERE rn = 1;
```

Asha / Carol / Ella. **PARTITION BY** restarts the numbering per department.

Q28. Running (cumulative) total of trade value by date.

```
SELECT trade_date, qty*price AS value,
       SUM(qty*price) OVER (ORDER BY trade_date, trade_id) AS running_total
FROM trades;
```

date	value	running_total
01-05	18000	18000
01-06	20000	38000
01-06	36400	74400
01-07	15750	90150
01-08	12150	102300

Q29. Day-over-day price change per symbol (LAG).

```
SELECT symbol, trade_date, price,
       LAG(price) OVER (PARTITION BY symbol ORDER BY trade_date) AS prev,
       price - LAG(price) OVER (PARTITION BY symbol ORDER BY trade_date) AS chg
FROM trades;
```

AAPL: 180 → 182 (chg +2). MSFT: 400 → 405 (chg +5). First row per symbol = NULL prev. **LEAD** does the same looking forward.

Q30. Each employee alongside their department's average salary (without collapsing rows).

```
SELECT name, dept, salary,  
       AVG(salary) OVER (PARTITION BY dept) AS dept_avg  
FROM employees;
```

This is the key win of window functions over GROUP BY: you keep all 6 rows *and* get the group aggregate on each.

Q31. Employees earning above their own department's average.

```
WITH dept_avg AS (  
  SELECT dept, AVG(salary) AS avg_sal FROM employees GROUP BY dept  
)  
SELECT e.name, e.dept, e.salary  
FROM employees e JOIN dept_avg d ON e.dept = d.dept  
WHERE e.salary > d.avg_sal;           -- Asha, Carol, Ella
```

A CTE (WITH) names the aggregate once and makes the query readable.

Q32. Walk the org hierarchy and show each person's level (recursive CTE).

```
WITH RECURSIVE org AS (  
  SELECT emp_id, name, manager_id, 1 AS lvl  
  FROM employees WHERE manager_id IS NULL      -- anchor: top boss  
  UNION ALL  
  SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 -- recursive step  
  FROM employees e JOIN org o ON e.manager_id = o.emp_id  
)  
SELECT name, lvl FROM org ORDER BY lvl;
```

Asha (1); Ben, Carol, Ella (2); Dev, Faisal (3)

Q33. CTE vs subquery vs temp table — when each?

- **Subquery** — one-off, inline, used once.
- **CTE** — readable, referenced multiple times in one statement, supports recursion. Not persisted.
- **Temp table** — materialized, reused across statements, can be indexed; good for multi-step ETL.

Part D — DDL/DML & Concepts (34–40)

Q34. Write DDL for a corporate_actions table with the right keys & constraints.

```
CREATE TABLE corporate_actions (
  ca_id      INT PRIMARY KEY,
  isin       CHAR(12) NOT NULL,
  ca_type    VARCHAR(20) NOT NULL
              CHECK (ca_type IN ('DIVIDEND','SPLIT','MERGER','RIGHTS')),
  ratio      DECIMAL(10,4) CHECK (ratio > 0),
  ex_date    DATE NOT NULL,
  record_date DATE NOT NULL,
  status     VARCHAR(12) DEFAULT 'ANNOUNCED',
  CONSTRAINT fk_sec FOREIGN KEY (isin) REFERENCES securities(isin)
);
```

Shows: PK, FK (referential integrity), CHECK (domain rules), DEFAULT, NOT NULL — exactly what a corporate-actions PO would design.

Q35. Increase price by 2% for all US-client trades. Then explain the safety step.

```
-- 1) SELECT first to confirm scope
SELECT * FROM trades WHERE client_id IN
  (SELECT client_id FROM clients WHERE country='US');
-- 2) Then UPDATE
UPDATE trades SET price = price * 1.02
WHERE client_id IN (SELECT client_id FROM clients WHERE country='US');
```

The safety habit: never run UPDATE/DELETE without first proving the WHERE with a SELECT — a missing WHERE hits every row.

Q36. Insert-or-update a client record (upsert).

```
-- Postgres/MySQL
INSERT INTO clients (client_id, client_name) VALUES (2,'Nova Global')
ON CONFLICT (client_id) DO UPDATE SET client_name = EXCLUDED.client_name;

-- ANSI MERGE (Oracle/SQL Server)
MERGE INTO clients c USING staging s ON (c.client_id = s.client_id)
WHEN MATCHED THEN UPDATE SET c.client_name = s.client_name
WHEN NOT MATCHED THEN INSERT VALUES (s.client_id, s.client_name);
```

Q37. DELETE vs TRUNCATE vs DROP — one example each and the key difference.

```
DELETE FROM trades WHERE trade_date < '2020-01-01'; -- rows, WHERE, rollback-able (DML)
TRUNCATE TABLE trades; -- all rows, fast, resets identity, no rollback (DDL)
DROP TABLE trades; -- removes the table entirely (DDL)
```

Only DELETE can be undone inside a transaction; TRUNCATE and DROP auto-commit.

Q38. Find and remove duplicate trades (same client, symbol, date).

```
-- Find
SELECT client_id, symbol, trade_date, COUNT(*)
FROM trades GROUP BY client_id, symbol, trade_date HAVING COUNT(*) > 1;

-- Delete duplicates, keep the lowest trade_id
DELETE FROM trades a USING trades b
WHERE a.trade_id > b.trade_id
  AND a.client_id=b.client_id AND a.symbol=b.symbol AND a.trade_date=b.trade_date;
```

Q39. Explain a transaction with a transfer, and which ACID property guarantees it.

```
BEGIN;
UPDATE accounts SET bal = bal - 100 WHERE id = 1;  -- debit
UPDATE accounts SET bal = bal + 100 WHERE id = 2;  -- credit
COMMIT;  -- both succeed, or ROLLBACK undoes both
```

Atomicity guarantees all-or-nothing. Consistency keeps balances valid, Isolation stops concurrent transfers interfering, Durability keeps the committed result after a crash.

Q40. What is the logical order of execution of a SELECT, and why can't you use a SELECT alias in WHERE?

Execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT .

```
-- Fails: alias 'notional' not yet created at WHERE stage
SELECT qty*price AS notional FROM trades WHERE notional > 20000;  -- ERROR
-- Fix: repeat the expression, or filter in HAVING/an outer query
SELECT qty*price AS notional FROM trades WHERE qty*price > 20000;
```

WHERE is evaluated before SELECT, so the alias doesn't exist yet. ORDER BY (last) *can* use the alias.